

Python Mastery: Object-Oriented Programming (OOP)

Classes, Objects, Inheritance, and Encapsulation

1. Introduction to OOP

Python is an object-oriented programming language. Almost everything in Python is an object, with its properties and methods.

Classes and Objects

A **Class** is like an object constructor, or a "blueprint" for creating objects. An **Object** is an instance of a class.

```
class MyClass:
    x = 5

p1 = MyClass() # Create an object named p1
print(p1.x)
```

2. The `__init__()` Function

To understand the meaning of classes we have to understand the built-in `__init__()` function. All classes have a function called `__init__()`, which is always executed when the class is being initiated.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("John", 36)
print(p1.name)
```

3. Methods and the self Parameter

Objects can also contain methods. Methods in objects are functions that belong to the object. The `self` parameter is a reference to the current instance of the class.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

4. Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class. **Parent class** is the class being inherited from. **Child class** is the class that inherits from another class.

```
class Student(Person):
    def __init__(self, name, age, graduation_year):
        super().__init__(name, age)
        self.graduationyear = graduation_year

x = Student("Mike", 19, 2024)
x.myfunc()
```

5. Encapsulation & Polymorphism

Encapsulation restricts access to methods and variables to prevent accidental modification. **Polymorphism** allows different classes to have methods with the same name.